LONGHORN SILICON · FEASIBILITY DOSSIER · REV 2.2 · SEPT 26, 2026 · STATE AT COMPUTE SHUTDOWN

Coral NPU on a ChipFoundry Shuttle

Google's **Coral NPU** (release `M3-2026-04-27`, `RvvCoreMiniAxi`) ported to sky130, **proven functionally correct at gate level**, sized against ChipFoundry's Double-Wide OpenFrame, and paired with a weight-streaming subsystem that was built, simulated and routed. Rev 2 replaces the synthesis-only estimates of rev 1 with measurements and adds a verdict on the MAC engine: **no VME; use a weight-stationary systolic array**. Every number carries its basis: MEASURED PROJECTED OPEN (the core route did not complete; see §08).

VERDICT
GO RVV + WS array

CORE LOGIC
6.58 mm² flat · 27/27 gate-level

MAC ENGINE
2.00 mm² WS vs 15.2 VME

WEIGHT STREAM
90.4 MB/s · 8.2 per pin

CORE COMPUTE
1.94 int8 MAC/cycle

CORE FMAX, TT POST-CTS
22 → 32.8 MHz with FPU fix

DIE USE (PLAN A)
89 % projected

00 What rev 2 changed

Rev 1 was synthesis-only on an 8 GB Mac. Rev 2 ran on a 28-core x86 host with the full open flow: Yosys, OpenROAD, Verilator gate-level simulation and multi-corner STA. Four of rev 1's headline numbers were wrong in ways that change the design. Rev 2.1 adds the full-core P&R diagnosis, an upstream FPU bug, and a ledger of which numbers are independently reproduced (§08).

CLAIM	REV 1	REV 2	CONSEQUENCE

Core netlist is correct	assumed	**4 bugs fixed**, then 27/27 programs pass at gate level with cycle counts identical to RTL	Rev 1's 7.34 mm² came from a netlist with a broken vector register file
Core int8 throughput	16 MAC/cycle, 1.6 GOPS	**1.94 MAC/cycle** best exact int8 GEMV	The core consumes about 97 MB/s of weights. One streaming port is enough
VME matrix engine area	5–7 mm²	**15.19 mm²** cells; 65% of it is the tile register file	A VME core does not fit the 32.2 mm² frame (§04)
Streaming complex	~90 kGE, 0.35 mm²	**154–237 kGE** (Yosys; 187–277 in ORFS); lean variant routed at 1.40–1.44 mm², 0 DRC; **40.7 MHz at ss**	Budget 1.4 mm², not 0.8
Second octal port	2× bandwidth, 180 MB/s	Useful only for derating, capacity and KV traffic	The core cannot consume 185 MB/s at 50 MHz
Prod build drops `TB_SUPPORT`	"slightly smaller"	The release RTL does not build without it; trace ports are pruned at the chip top instead	The chip top leaves 11,286 trace bits unconnected
Core closes 50 MHz "with margin"	assumed	**22 MHz** tt after CTS, as released. The FP32 FMA is built with **0 pipeline stages** (upstream config bug)	A one-line fix restores the 3-stage FMA; it is gate-level verified and in P&R now (§08)
Weight stream per port	~90 MB/s	**90.4 MB/s** (lean, routed design), 8.2 MB/s per pin; **73.5** at the ss aclk of 40 MHz	Re-simulated by the lead; the core's ceiling scales with the same aclk, so one port stays matched

01 The silicon on offer

ChipFoundry shuttles **SKY130 only**. The 32 mm² option is the **Double-Wide OpenFrame**: a bare padframe with no management SoC.

PLATFORM	USER AREA	GPIOS	HARNESS	FIT FOR THIS DESIGN
Caravel	10.3 mm²	38	Mgmt RISC-V SoC, housekeeping SPI, Wishbone, 128 LA probes	No. The core alone projects to 16.6 mm² placed
Caravan	~10 mm²	27 + 11 bare	Caravel with analog pads	No
OpenFrame	~15 mm²	44	Bare padframe	No. Core + TCM + streaming projects to 21.3 mm²
Double-Wide OpenFrame PICK	**32.2 mm²**	**63**	Bare padframe, die 7,176 × 5,188 µm, user 6,754.63 × 4,766.63 µm, all pads `sky130_ef_io__gpiov2_pad`	Yes, with the WS array (plan A, 89%)

Commercials: **$14,950** per standard slot; double-wide is quoted by sales (budget about two slots). **CI2612 commits Oct 8, 2026** (tapeout Dec 7, silicon May 25, 2027).

02 The core, verified

The release is **RV32IMF_Zve32x**: a 4-way in-order scalar core, an RVV 1.0 backend at VLEN 128 with a flop-based 32 × 128 b register file, an fpnew FPU, and two 128-bit AXI4 ports. ITCM is 8 KB (one 512 × 128 SRAM) and DTCM is 32 KB (one 2048 × 128 SRAM in the release RTL). Both are bound to SRAM22 macros through a sky130 `Sram` module, and the clock gate to the `sky130_fd_sc_hd__dlclkp` ICG.

Five bugs a synthesis-only study would have taped out

WHERE	WHAT WAS WRONG	HOW IT WAS CAUGHT	FIX
`rvv_backend_vrf`	A dynamic-index `always_comb` inside a generate loop became 1,920 latches and 18,432 multiply-driven bits after sv2v → Yosys. The vector register file	Yosys driver-conflict warnings	Static per-register decode. Equivalent to the original over 3 × 2 M random

	write path was wrong in gates		cycles, all ports and full register state
`RvvCore`	**Upstream bug:** `rvv2rvs_frd_ready[3:2]` undriven into the retire logic. Verilator zero-initializes and hides it	Yosys "used but has no driver"	Tie to 0, like slot 1
`RvvFrontEnd`	`lmul_orig` left out of the async reset, which forces an async-load flop sky130 cannot map cleanly	Yosys "async reset value is not constant"	Reset it (dead state after reset anyway)
`Sram_sky130.v`	**Our own binding:** DTCM bank select used `addr[9:8]` instead of `addr[10:9]`, so the upper 16 KB aliased onto the lower half	Gate-level sim: full-TCM fill test, GEMV and memcpy failed	Corrected before any P&R result was used
`FloatCoreWrapper`	**Upstream bug:** `PipeRegs: '{default: 'd3}` on a *packed* [opgroup][format] array zero-extends, so FP32 gets 0 pipeline registers. The release builds a single-cycle FP32 FMA: 28 ns on sky130, the core's critical path	Critical-path dissection after CTS; confirmed by FPU flop count (3 vs 761) and FMA latency (1 vs 3 cycles)	`'{default: '{default: 32'd3}}`. 27/27 gate-level, cycle-identical to the fixed RTL. Scalar `fmadd.s` throughput drops from 1 per 2 cycles to 1 per 5; vector and int8 GEMV unchanged

Functional proof

MODEL	PROGRAMS	RESULT
Release RTL, Verilator (golden)	27	27/27. Two newer upstream tests fail as expected: they reproduce bugs Google fixed after M3 (FRM hazard 511de1e7, mstatus FS/VS 9c741cd7)
Release RTL, random X-init, 10 seeds	27	Pass, cycle counts identical

sky130 gate netlist (coral_npu_top, SRAM22 + ICG cells)	27	**27/27, cycle counts identical to RTL**, including AXI slave bandwidth and DMA contention runs

Programs: scalar I/M/F (341 checked results), vector FP including `vfmv.f.s`, three int8 GEMV kernels against a host golden, two vector memcpys, AXI master-port latency tests, a full ITCM + DTCM fill and read-back, and 16 upstream programs.

Area

BLOCK	CELL AREA	KGE	FLOPS	BASIS
coral_npu_top, flattened (P&R input)	**6.58 mm²**	1,753	58,848	VERIFIED 840,452 cells; two independent mappings agree to 0.05%. Flattening prunes 6,179 trace-only flops
coral_npu_top, hierarchical	7.39–7.58 mm²	1,970–2,019	65,027	MEASURED No cross-boundary pruning; this is the netlist simulated at gate level
SCore	1.04 mm²	276	7,152	MEASURED
rvv_backend_mulmac	0.63 mm²	169	–	MEASURED
FloatCoreWrapper	0.09 mm²	25	–	MEASURED
Placed core + 5 TCM macros, one block	16.4 mm²	–	–	MEASURED 4,200 × 3,900 µm, 49% cell utilization, 10.56 mm² after placement repair; global route congested (§08)

gf180mcu, for reference: the same netlist is 22.32 mm² on gf180mcu_fd_sc_mcu7t5v0 (about 3× sky130 HD), and gf180 is not currently shuttled. sky130 HD is the only sensible node here.

Cross-check against a commercial TSMC 65 nm synthesis

A colleague synthesized the upstream Bazel emits on TSMC 65 nm at 250 MHz. Flop counts do not depend on the library, so they are the fair comparison. Ours agree within 4%.

TOP	TSMC65 INSTANCES	TSMC65 FLOPS	TSMC65 AREA	SKY130 FLOPS	SKY130 AR
CoreMiniAxi	96.776	11.430	0.35 mm² ·	–	

			242 kGE		
RvvCoreMiniAxi	984,016	67,640	3.16 mm² · 2.19 MGE	65,027	7.39 m (6. fla tra prune
VmeCoreMiniAxi	2,139,687	138,110	7.03 mm² · 4.88 MGE	≈139,100	≈22.6 m

TSMC65 worst slack at 4 ns: 0 / −31 / −52 ps. The sky130 VME figure is M3 core + measured engine; the full VME core was emitted from Bazel but its flat synthesis did not finish within the host's memory cap.

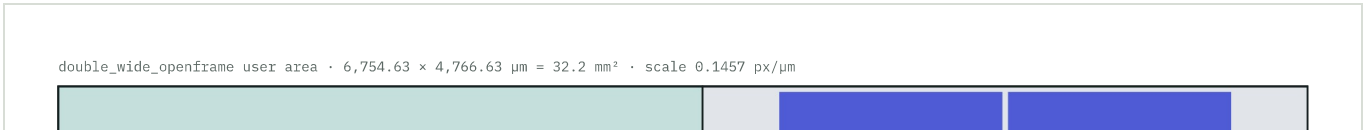
P&R flow, proven on a TCM stand-in

Before spending hours on the full core, the exact floorplan, macro placement and power grid were run through ORFS on a stand-in holding the real five SRAM22 macros. SRAM22's power pins are met2 only, so the macro grid adds met3 straps across each macro. Result: 773 met2–met3 vias per macro, both nets fully connected (`check_power_grid`), **0 DRC after detailed route**, and the SRAM read path closes at ss with 5.25 ns slack at 50 MHz (tt 9.72 ns, ff hold +0.008 ns). The full-core run uses the same scripts.

03 Mapped to the floorplan

Both configurations are drawn to scale on the Double-Wide OpenFrame user area. Block regions are cell area × 1.35 (P&R growth measured on the routed streaming complex) ÷ 0.60 utilization. Macros are drawn at LEF size. Check against run 1 (§08): the core and its 5 TCM macros were placed in a 16.4 mm² block at 49% cell utilization, and that block is congested in global route. The 16.6 mm² core region here is the right size, but it has to hold the TCM macros too. Budget about 20 mm² for core + TCM until the route closes.

Plan A: M3 RVV core + weight-stationary array (fits, 89%)



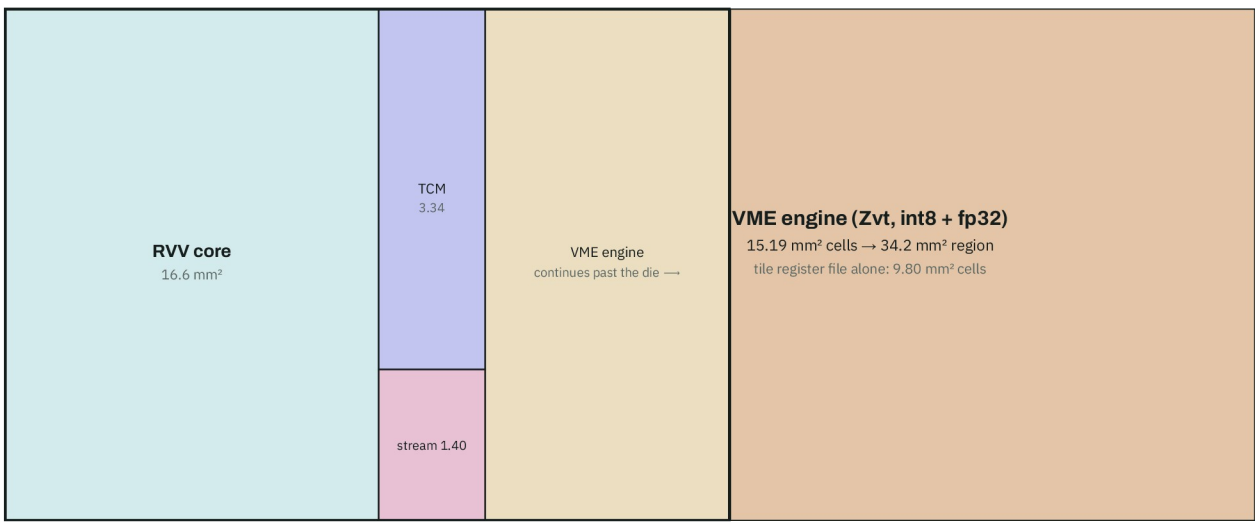


tiling check: 16.60 + 6.61 + 4.50 + 1.40 + 1.50 + 1.58 = 32.19 mm² · regions abut

Eight macro slots in two columns: four DTCM banks, ITCM, the WS array's two private buffers, and a spare. Each buffer is one 128-bit SRAM22, so the array gets one 16-byte weight row per cycle without contending with the core for DTCM. Region areas are projections from measured cell area; macros are exact.

Plan B: M3 RVV core + VME matrix engine (does not fit)

same user area, scale 0.0845 px/µm · blocks packed left to right at full die height



die edge · 6,754.63 µm 23.3 mm² beyond the die (172% of user area)

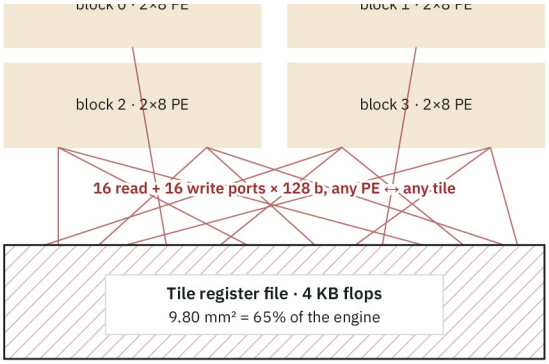
The VME core needs 55.5 mm² at the same assumptions. Tighter packing does not rescue it: even the int8-only engine at 70% utilization projects to 37.1 mm² for core + engine alone, before TCM and streaming. The in-tree VME configuration also defaults DTCM to 128 KB, which adds 11 more SRAM22 macros.

CONFIGURATION	CELL / MACRO AREA	PROJECTED REGION	SHARE OF 32.2 MM²	STATUS
RVV core + 5 TCM macros + lean streaming	6.58 + 2.82 + 0.70	21.3 mm²	66%	FITS
+ WS 16×16 array + 2 buffer macros + mgmt SoC (plan A)	+ 2.00 + 1.13	**28.7 mm²**	**89%**	FITS
RVV core + VME int8-only + TCM + streaming	+ 11.85	48.0 mm²	149%	DOES NOT FIT
RVV core + VME int8 + fp32 (upstream default), plan B	+ 15.19	55.5 mm²	172%	DOES NOT FIT

04 The MAC engine: tiled register file vs weight-stationary array

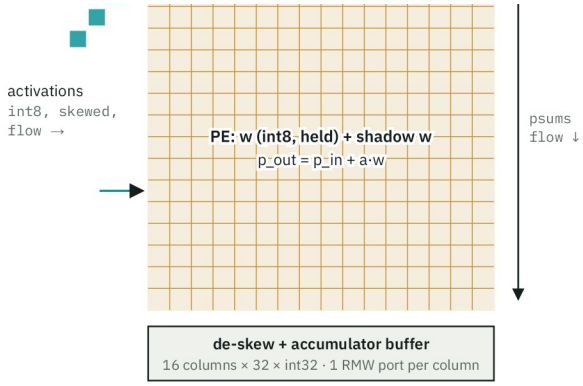
Both engines deliver 256 int8 MACs per cycle. They differ in where partial sums live. VME keeps them in a central 4 KB register file that every PE reads and writes every cycle, which needs a 16-read, 16-write crossbar. A weight-stationary array pins the weights in the PEs and lets partial sums flow down the columns to accumulators at the edge, so every wire is nearest-neighbour. The wiring is what separates 15.2 mm² from 2.0 mm².





Every PE reads and writes its partial sums through the crossbar every cycle. Storage is 1.19 mm²; read muxes 4.12 mm²; write muxes 4.49 mm².

15.19 mm² · 74,077 flops · 1.69 GOPS/mm²



Nearest-neighbour wiring only. The tile switch travels as a wavefront with the first activation of the next tile, so there is no drain bubble.

2.00 mm² · 32,598 flops · 12.8 GOPS/mm²

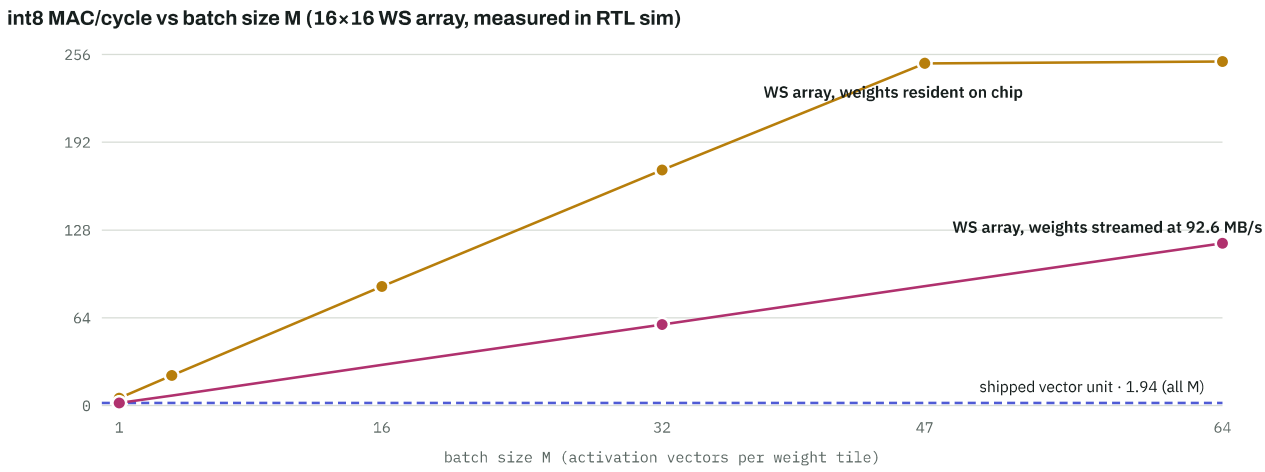
Areas are sky130 HD tt cell area from Yosys + ABC at 20 ns, same recipe for both engines. The VME figure is the upstream default datatypes (int8 + fp32); int8-only VME is 11.85 mm².

	VME / ZVT (UPSTREAM RTL)	WEIGHT-STATIONARY 16×16 (THIS STUDY)
int8 MAC/cycle	256 (4 blocks × 2×8 PEs × dot-4)	256 (16 × 16 PEs × 1)
Partial-sum storage	4 KB flops, 16R + 16W × 128 b, fully addressable	2 KB flops (16 × 32 × int32), one sequential RMW per column
Cell area, sky130 tt	**15.19 mm²** (11.85 int8-only)	**2.00 mm²** (2.74 with 64-deep accumulators; array alone 1.48)
Flops	74,077	32,598
int8 GOPS/mm² at 50 MHz	1.69 (2.16 int8-only)	**12.8**
Timing at 50 MHz	fp32 accumulate misses by 1.59 ns; int8 ≥ 10 ns slack	**Routed, signed off with extracted SPEF:** 49.8 MHz tt, **24.0 MHz ss** (WNS −21.7 ns at 20 ns), 61.8 MHz ff. Hold violations remain at tt (−0.10 ns) and ff (−0.17 ns). 0 DRC, 0 antenna, 3.65 mm² block at 61% utilization
Datatypes	int8, fp32; int16 and bf16 behind flags no upstream build sets; no fp16	int8 × int8 → int32; int4 weights by nibble unpack
Verification	upstream tests	Verilator vs host golden: 11 configurations, back-to-back tile switching, worst-case −128 × −128, two

injected mutations both caught

Utilization depends on batch size

A weight-stationary array only pays off when each loaded weight tile is reused across many activation vectors. The measured steady state is M / max(M, 47) for a 16×16 array with this controller. The second limit is the pins: with weights streamed from off-chip at 92.6 MB/s, each weight byte supports at most M MACs, so throughput can never exceed 1.85 · M MACs per cycle.



WS array, weights resident (measured) WS array, weights streamed (bound: 1.85 · M)

shipped vector unit (measured best kernel)

Points are from Verilator runs of 32 back-to-back K-tiles at each M, all results checked against a golden model. At M = 1 every engine collapses to the pin rate, so the array brings nothing to batch-1 decode.

What it buys for an LLM on this chip

WORKLOAD (10 MB INT8 MODEL)	VECTOR UNIT ONLY	WITH WS ARRAY	GAIN	BOUND BY
Decode, 1 token (M = 1)	≈ 114 ms · 8.8 tok/s	≈ 114 ms · 8.8 tok/s	1×	Weight stream for both: 88 MB/s end to end, measured (§07)
Prefill, 64-token prompt	6.6 s	**0.11 s**	**≈ 61×**	Vector compute vs weight stream

				(weights read once for all 64 tokens)
Always-on CNN, weights resident (5 M MAC)	52 ms	≈ 0.5 ms	≈ 100×	Vector compute vs array compute
Batched decode, 8 streams	8 × 108 ms	≈ 108 ms for all 8	≈ 8×	Weight stream shared across the batch

> **Recommendation.** Tape out the M3 RVV core with a 16×16 weight-stationary int8 array, not VME. The array is 7.6× smaller at the same MAC rate, closes on sky130 with nearest-neighbour wiring, fits the frame with 11% to spare, and turns prompt prefill from seconds into about 100 ms. Attach it to the AXI fabric with its own two SRAM22 buffers so the DMA fills its weight buffer directly and it never competes with the core for DTCM. Decode speed stays pin-bound at about 9 tok/s for a 10 MB model with either engine; only more pins or a faster aclk move that.

05 Compute ceiling of the shipped core

The vector multipliers can retire 16 widening MACs per cycle, but a GEMV never gets there. Measured on the release RTL with microbenchmarks and 8 hand-tuned kernels, all exact against a host golden:

KERNEL	INT8 MAC/CYCLE	WEIGHTS CONSUMED AT 50 MHZ
Row-wise GEMV (first version)	1.18–1.29	64 MB/s
vwmul m4→m8 + vwredsum, 4 rows share x	1.74–1.85	92 MB/s
asm: vle8 + vsext + vwmacc e16→e32, 4 rows, loads interleaved	**1.94**	**97 MB/s**
asm: int4 weights, packed 2 per byte	1.99	50 MB/s

- **Vector loads go through the scalar LSU** one at a time, at 16 bytes per 4 cycles: about 3.9 B/cycle at LMUL 8.
- **I**

- **Issue is below one vector instruction per cycle** (about 0.75) through the in-order front end, into an 8-entry ROB.
- **Exact int8 → int32 needs two arithmetic instructions per 16 MACs**, because two worst-case int8 products overflow int16.

So the core and one streaming port are matched at about 95 MB/s. The WS array does not raise this ceiling for batch-1 work; it adds a second compute path for work that reuses weights.

06 TCM macros

The TCMs need 128-bit, 1RW, byte-masked SRAM with registered reads. `sram22_512x128m4w8` (UC Berkeley SRAM22, via ChipFoundry) is a native match: one macro for ITCM, four banks for DTCM. Liberty clock-to-dout: 5.3 ns tt, **9.3 ns ss**, 3.6 ns ff, so it closes at 50 MHz on every corner (5.25 ns slack at ss in the routed stand-in).

PLAN	MACRO	ITCM + DTCM	MACROS	STATUS
A PICK	`sram22_512x128m4w8`	2.82 mm²	5	ROUTED PDN, pin access and ss timing proven in the stand-in
B	`CF_SRAM_1024x32`	1.41 mm²	12	FALLBACK densest; more macros to place
C	`OpenRAM 1rw1r 32×512`	5.69 mm²	20	TIMING RISK

07 Streaming weights without a DDR PHY

The streaming complex was built from permissive pulp IP: 2 HyperBus controllers, 2 iDMA engines, an AXI mux, and a 32-tap `dlygate4sd3` RWDS trim line. It was simulated against a datasheet HyperRAM model and routed on sky130. No permissive octal/Xccela controller exists, so HyperBus stands in for the octal port; the framing differs only in the command/address phase.

Pads, from the real liberty

SKY130_EF_IO__GPIOV2_PAD, SS 100 °C, STRONG DRIVE, FAST SLEW	VDDIO 3.0 V	VDDIO 1.65 V
Core → pad, 5–15 pF	6.2–8.0 ns	14–21 ns
Pad → core	2.0–2.7 ns	4.4–5.5 ns
Pad edge, 20–80%	2.4–3.7 ns	5.3–9.1 ns

INTERFACE	MAX SCK AT SS	CONDITION
QSPI SDR	25 / 50 / 81–104 MHz	half-cycle / opposite-edge / calibrated delayed capture
HyperRAM or 3 V octal, DDR + strobe	≈ 100 MHz (10 pF) · 81 MHz (15 pF)	3.0 V I/O; limited by pad slew and clock duty
1.8 V octal or HyperRAM	33–38 MHz	gpiov2 at 1.65 V is too slow; **use 3 V memory parts**

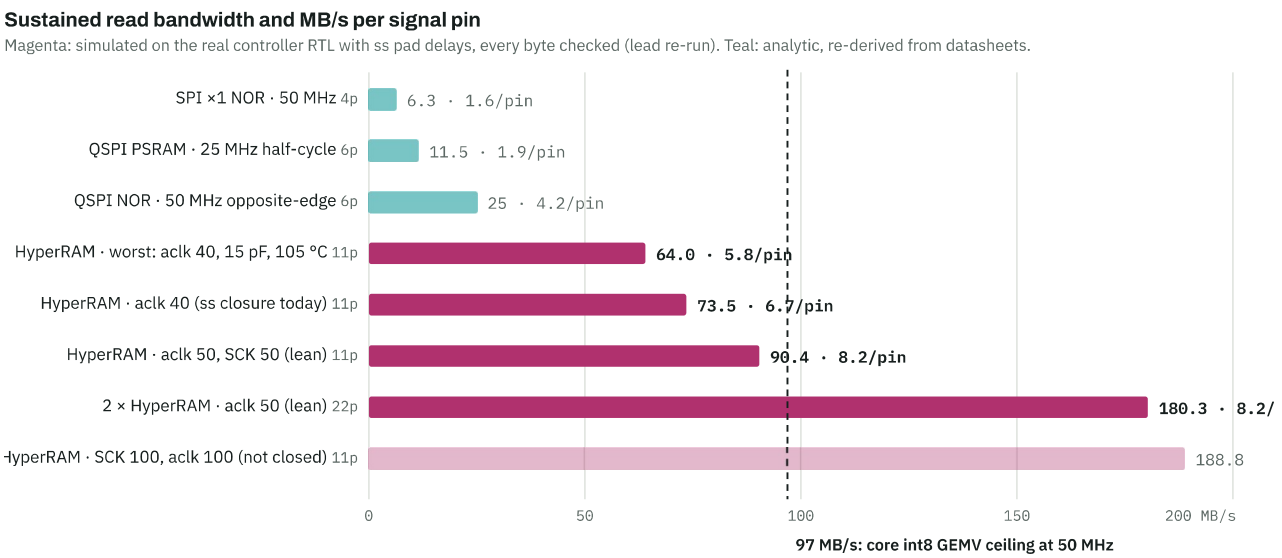
Measured bandwidth (RTL sim, 64 KB DMA transfers, every byte checked)

CASE	MB/S PER PORT
Stock complex, SCK 50 MHz, fixed 2× latency (CA 1.5%, latency 2.5%, CS# gaps 2.4%)	92.6
Lean complex (the routed one), aclk 50, SCK 50, 10 or 15 pF	**90.4**
Lean, aclk 40 MHz (what the complex closes at ss today)	73.5
Lean worst case: aclk 40, 15 pF, 105 °C part (tCSM 1 µs)	64.0
Lean, aclk 25 MHz (≈ ss signoff clock of a tt-50 MHz design), SCK 50	47.4
Lean, SCK 10 / 20 / 25 / 30 MHz (aclk 50)	13.4 / 32.5 / 42.4 / 52.0
Same, variable latency (85 °C part)	94.0
tCSM 1 µs (105 °C part)	72–76
SCK 66–100 MHz, aclk 50 MHz	96–97 (aclk-limited)

aclk 100 MHz, SCK 83 / 100 MHz	157 / 189
2 ports concurrently, default DMA settings	48.5 each
2 ports, decoupled R/W, 64 B write bursts	92.6 each (185 total); lean 90.2 each (180.3), 73.3 each at aclk 40
QSPI NOR / PSRAM at 50 MHz	24–25
EF_QSPI_XIP_CTRL as shipped	8.7

Bandwidth per pin, hardened

The lead re-derived this table independently of the streaming engineer. The pad delays were re-parsed from the PDK's sky130_ef_io__gpiov2_pad_ss_ss_100C_1v60_3v00 liberty: 7.06 ns core→pad at 10 pF and 2.39 ns pad→core, matching within 0.15 ns. The HyperBus rows were re-simulated (0 byte errors, 0 tCSM violations), and the QSPI arithmetic was redone from the datasheets. Pins count signals only.



Pins count signals only (HyperRAM: CK, CS#, RWDS, DQ0–7; add RESET# for 12). The lean complex is the one placed and routed; the stock complex sustains 92.6 MB/s. A port moves at most 2 bytes per aclk cycle and sustains 1.81–1.84, so its rate scales with aclk. The core's ceiling scales the same way (1.94 B per cycle), so one port stays within 5–7 % of the core at any aclk. At 50 MHz SCK one port loses 1.6 % to command/address, 2.6 % to latency, 4.3 % to CS# gaps and 1.1 % to DMA overhead. 50 MHz QSPI needs opposite-edge capture and at most 10 pF at ss.

INTERFACE	SIGNAL PINS	CLOCK AND CAPTURE	MB/S	MB/S PER PIN	BASIS

SPI ×1 NOR	4	50 MHz, opposite-edge	6.3	1.6	analytic
QSPI PSRAM APS6404L	6	25 MHz, half-cycle	11.5	1.9	analytic, re-derived
QSPI NOR W25Q128JV	6	50 MHz, opposite-edge, ≤10 pF at ss	25.0	4.2	analytic, re-derived
HyperRAM 3 V, lean complex	**11**	SCK 50 DDR, RWDS capture, aclk 50	**90.4**	**8.2**	simulated, reproduced
HyperRAM 3 V, aclk 40 (ss today)	11	SCK 50	73.5	6.7	simulated (lead)
HyperRAM 3 V, worst case	11	aclk 40, 15 pF, 105 °C part	64.0	5.8	simulated (lead)
2 × HyperRAM, lean	22	aclk 50	180.3	8.2	simulated, reproduced

Why one port stays matched at any clock. A port sustains 1.81–1.84 bytes per aclk cycle (the controller's cap is 2). The core's best GEMV consumes 1.94 bytes per cycle. Both run on the same aclk, so at 40 MHz the pair is 73.5 vs 77.6 MB/s and at 50 MHz 90.4 vs 97. HyperBus gives about 2× the bandwidth per pin of QSPI. One HyperRAM plus a QSPI boot NOR uses 17 of the 63 GPIOs.

Two correctness bugs in the open IP, both fixed. The pulp HyperBus PHY starts the next command while read data is still in flight after a tCSM chop under back-pressure, which overflows the RX FIFO and corrupts data. Separately, sv2v + Yosys turn 66 bits of `axi_dw_downsizer` into latches and constant X. Verilator on the original source shows neither.

Four ports and a 100 MHz bus clock

4 CONCURRENT HYPERRAM PORTS (DECOUPLED R/W, 64 B WRITE BURSTS)	CLOSES TIMING?	TOTAL MB
aclk 100 / SCK 100	No, at any corner; SCK 100 also needs external quadrature	728–7!

	clocks	
aclk 96 / SCK 78	tt, with a two-cycle DMA burst splitter and 128-bit controller interface	58
aclk 61 / SCK 51.5	ss, with the same fixes (2-port netlist)	38
aclk 40 / SCK 50	**ss, current netlist**	**293–30**

> **What four ports cost and buy.** Each port's controller is the limit; four ports scale exactly 4×, and the 128-bit fabric runs at 47 % of capacity. The shared write channel is a trap: with default DMA settings (4 KB write bursts) four ports deliver only 80–201 MB/s in total. Four ports are 386–422 kGE, 1.7–1.8 mm² of cells, and 44 + 1 pins, leaving 58 of 63 GPIOs used with QSPI boot, clock, JTAG and UART. The core consumes about 97 MB/s at 50 MHz, so ports past the first pay off only with a faster weight consumer such as the WS array on batched work. The lead reproduced 728 MB/s (100/100) and 293 MB/s (ss clock) with 0 byte errors.

End to end: pins → DMA → DTCM → GEMV, measured

One Verilator simulation runs the whole loop. Weights stream from a HyperRAM model through the lean HyperBus + iDMA complex into the NPU's DTCM ping-pong halves (8 KB tiles). The FPU-fixed Coral core runs the fastest exact int8 GEMV kernel on the other half, and every output is checked against a golden model: 0 errors in 8 runs. The lead re-ran all 8 and got cycle-identical results.

SHAPE (TINYSTORIES)	ACLK	MAC/CYCLE	WEIGHT MB/S	% OF PORT LIMIT	DMA BUSY	CORE
attention 512×512	50	1.73	86.3	95.5%	96.8%	
FFN 512×2048	50	1.77	88.3	97.7%	99.0%	
FFN 2048×512	50	1.76	88.2	97.6%	98.9%	
LM-head slice 1024×512	50	1.75	87.6	96.9%	98.2%	

FFN 512×2048	40	1.80	72.1	98.1%	99.0%

TINYSTORIES-28M, BATCH-1 DECODE	MB PER TOKEN	50 MHZ	40 MHZ
vocab 50,257 (8 layers ≈ 25.2 MB + LM head 25.7 MB + KV 2.1 MB at context 256)	53.0	**1.65 tok/s**	1.35 tok/s
vocab 8,192	31.5	**2.78 tok/s**	2.27 tok/s

> **The stream binds, and the core is close behind.** The DMA is in flight 97–99 % of the time, while the core runs at 1.73–1.80 MAC/cycle against its 1.94 ceiling and waits 2.5–11 %. So pins and core are balanced within about 10 %, and any added per-token work (non-GEMV ops, int4 unpacking) makes the core the limit. A second port buys at most about 10 % for this core. Loss against the port alone is 2.4 %: 1.5 % pipeline fill and drain, 0.3 % tile relaunch.

Capacity per device class

CLASS	REPRESENTATIVE PART	CAPACITY	ROLE
QSPI PSRAM	APS6404L-SQ	8–16 MB	Cheap activations tier
HyperRAM (3 V)	S27KL0642 / S80K	8–64 MB	**Weight tier + KV cache; controller verified here**
Octal xSPI PSRAM	APS256XX	32–64 MB	Same role; needs a CA-format change to the pulp PHY
QSPI NOR	W25Q128JV → MT25QL02G	16–256 MB	Bulk read-only weights, boot
Octal NOR	MX25LM51245G	64–128 MB	Bulk store if NOR is the primary tier

What a decode token costs

Batch-1 decode reads every weight once per token: tok/s ≈ bandwidth ÷ model bytes, as long as the core keeps up. At 50 MHz it does up to about 97 MB/s (§05), so the 180 MB/s column needs aclk raised to about 100 MHz or the WS array running batched decode.

MODEL (INT8 BYTES)	24 MB/S · 1× QSPI	90.4 MB/S · 1× HYPERBUS	180 MB/S · 2× HYPERBUS

10 MB (~10 M params)	2.4 tok/s	9.0 tok/s	18.0 tok/s
32 MB (one 32 MB part)	0.75	2.8	5.6
100 MB	0.24	0.90	1.80
500 MB (NOR-resident)	0.05	0.18	0.36

How much on-die SRAM

More SRAM does not raise decode token rate, since every weight still crosses the pins once per token. It buys descriptor slack, residency for small networks, and room for the WS array's prefill activations.

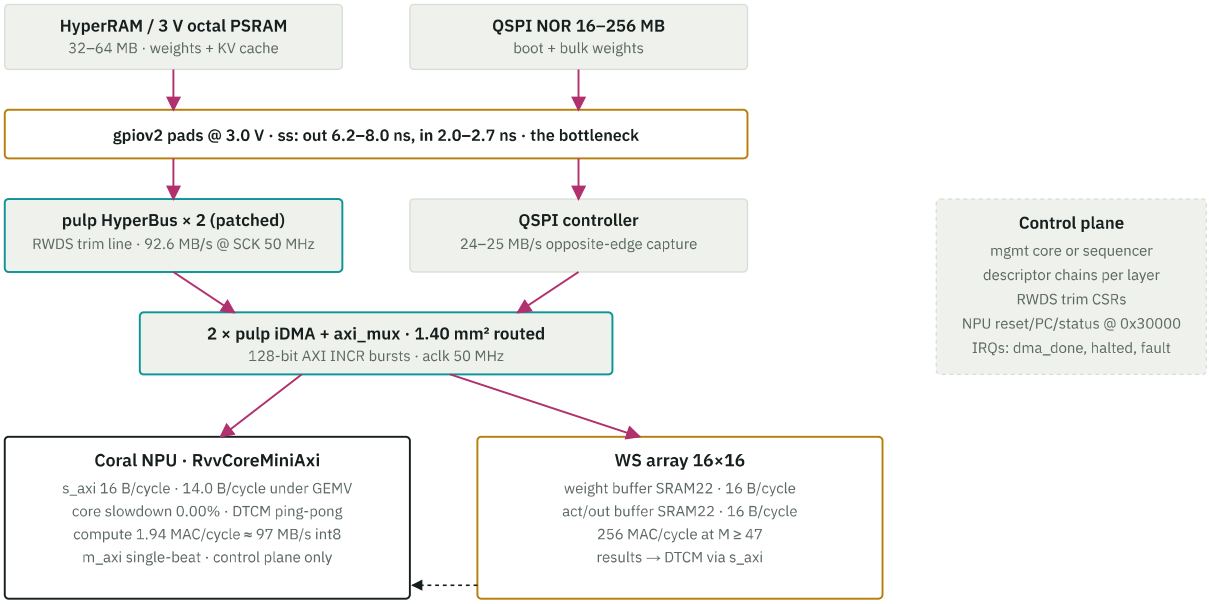
DTCM	COST (CF_SRAM / SRAM22)	PING-PONG HALF	FILL AT 92.6 MB/S	WHAT IT ENABLES
32 KB (shipped)	0.94 / 2.26 mm²	16 KB	177 µs	Streaming works; the sequencer must meet 177 µs deadlines
40 KB (plan A spare slot)	– / +0.56 mm²	20 KB	221 µs	Uses the free macro slot in plan A
128 KB	3.9 / 9.0 mm²	64 KB	0.71 ms	Whole layer blocks resident; does not fit plan A alongside the WS array unless CF_SRAM is used

The KV cache cannot live on die: at 8 layers × d_{kv} 512 int8 it costs about 8 KB per token of context, 8 MB at 1 k tokens. It lives in external PSRAM and adds roughly 10–20% bus traffic on attention layers.

DMA into the NPU, measured

- **AXI slave port:** 16 B/cycle (800 MB/s) for any burst length with more than one burst in flight.
- **Contention:** the core has fixed priority on the TCMs. A DMA running beside GEMV slows the core by **0.00%** and still gets 14.0 B/cycle (701 MB/s), 7.6× what the pins deliver.
- **AXI master port:** single-beat only (len 0), 1 read + 1 write outstanding, 16 B/(latency + 5) cycles. Vector loads use 16-byte beats, which contradicts the integration guide's "1, 2 or 4 bytes". The master port cannot stream weights; the DMA pushes them in.

Weight path with measured rates



Every stage downstream of the pads runs faster than the pads: 92.6 MB/s per port at the pins, 800 MB/s on the fabric and the NPU slave port. A second port helps for 105 °C derating, capacity and KV-cache traffic, and only reaches 185 MB/s total with decoupled read/write DMA settings.

08 Verification and place-and-route

The netlist that goes into P&R is the one that passed gate-level simulation. We mapped the patched RTL to sky130 HD cells, with SRAM22 and `dlclkp` kept as cells, and ran 27 programs on the gates in Verilator. The programs cover scalar integer, M and F, vector FP, three int8 GEMVs, vector memcpy, master-port latency, a full TCM fill and 16 upstream tests, and they check every result against a host golden. All 27 pass with cycle counts identical to the release RTL. The netlist was then flattened, which removes the dead trace logic, and handed to OpenROAD-flow-scripts unchanged.

unchanged.

The five RTL bugs and their fixes are listed in §02; the two IP bugs in §07.

Run 1: the release as-is

STAGE	RESULT
Input	840,452 cells, 6.58 mm², 58,848 flops, 5 × SRAM22
Floorplan	Block 4,200 × 3,900 µm = 16.4 mm². Three macros across the top in R0 (pins face the logic), two across the bottom in MX. 58 % utilization with macros, 49 % cells only
Power grid	met1 rails and a met4/met5 mesh. SRAM22 pins are met2 only, so each macro gets horizontal met3 straps: 773 met2→met3 vias per net per macro, and `check_power_grid` passes (routed stand-in)
Placement	Timing-driven global placement took 87 min and added 6.2 % area. After resizing the design is 10.56 mm² (65 %). SRAM22 `dout` pins cannot be slew-repaired because the macro's output driver is too weak for the wire
After CTS (tt)	**WNS −25.45 ns at 20 ns; minimum period 45.45 ns = 22 MHz**
Global route	**Did not converge.** After 6 h the router had cycled through 4+ rounds of congestion-removal iterations and dropped the clock nets' non-default rules. Stopped; the as-released core + TCM does not route cleanly in 16.4 mm² at 49 % cell utilization

> **Why 22 MHz.** The critical path is 49.7 ns: 28.1 ns in the scalar FPU, 12.9 ns in the RVV front end's `vsetvl` config chain, and 8.8 ns across the scalar register file, dispatch and FP register file. The FPU part is the upstream pipeline bug in the table above. With FPU paths masked, the next limiters are the vector FMA stage (41 ns register to register, −16.7 ns), the synchronous-reset fan-out from the CSR reset register (−14 ns) and the retirement buffer (−6.9 ns). DTCM inputs miss by 2.4 ns; everything else is within 0.3 ns. Two limits of this run apply. The netlist was mapped for area, with 119k weak `_0` cells, and the sky130 platform restricts repair to same-footprint swaps. Run 2 lifts the second.

> **Signoff corner.** On this PDK the ss corner (100 °C, 1.60 V) is about 2× slower than tt for logic: the routed WS array is 49.8 MHz tt but 24.0 MHz ss, and the streaming complex 78 MHz tt but 40.7 MHz ss. SRAM-bound paths degrade less (TCM stand-in 97.3 → 67.8 MHz). Plan for signoff clocks near half of the tt figure. Stream and core scale together. Simulated at an aclk of 25 MHz (SCK 50), one port gives 47.4 MB/s against a core ceiling of 48.5 MB/s: about 4.7 tok/s on a 10 MB

> model, guaranteed across process and temperature.

Runs 2 and 3: FPU fix and full-strength repair

Same flow on the gate-verified FPU-fixed netlist (6.61 mm², 59,658 flops, 27/27 at gate level), with `MATCH_CELL_FOOTPRINT=0`, last-gasp repair on, and a 4,400 × 4,000 µm block to relieve the congestion seen in run 1. The fix costs scalar FP throughput: a scalar `fmadd.s` goes from 2 to 5 cycles, because the scalar float core does not overlap FP ops. Vector FP and the int8 GEMV are unchanged.

STAGE	RUN 1 (AS RELEASED)	RUN 2 (FPU FIXED)
Block	4,200 × 3,900 µm	4,400 × 4,000 µm
After placement repair: WNS / TNS	−33.9 / −45,370 ns	−14.0 / −4,141 ns
After CTS: WNS → Fmax (tt)	−25.45 ns → **22.0 MHz**	−10.48 ns → **32.8 MHz**
Design area after CTS	10.82 mm² (67%)	10.77 mm² (62%)
Global route	did not converge (6 h)	did not converge (7 h, 7 rounds)
Run 3: 4,800 × 4,400 µm (21.1 mm²), density 0.50, same FPU-fixed netlist	—	CTS: −10.95 ns → **32.3 MHz** tt; global route **did not converge (13 h, 12 rounds)**

The FPU fix is worth +49 % Fmax at tt and removes 93 % of the total negative slack. The remaining −10.5 ns sits in the vector FMA stage and the synchronous-reset fan-out. Applying the ~2× ss derate measured on the WS array and the streaming complex, expect roughly 16–20 MHz at ss for this netlist until those two are pipelined. Routed numbers follow.

More FMA stages do not help on an open flow. We synthesized the vector FMA standalone with 3, 4 and 5 pipeline stages. Wire-free, it reaches 36.0 / 38.5 / 36.1 MHz at tt and 17.3 / 19.2 / 17.9 MHz at ss, and each stage adds about 12 % area. fpnew places its pipeline registers at fixed input, middle and output positions and relies on synthesis *retiming* to move them into the multiply-add. Yosys, ABC and OpenROAD do not retime; ABC's sequential mode made it worse (22.5 MHz). Reaching 50 MHz tt needs hand-placed registers inside `fpnew_fma`, or a commercial synthesis flow with retiming.

Which numbers are hardened

NUMBER	VALUE	STATUS	HOW IT WAS CHECKED
Gate netlist function	27/27, cycle-identical	VERIFIED	RTL vs gates, same stimulus; VRF patch equal over 6M random cycles; 13 modules lock-step equal
Core cells, flat	6.58 mm², 58,848 flops	VERIFIED	Two independent mappings agree (6.581 vs 6.584 mm²); flops match TSMC 65 within 4 %
Pad delays (ss, 3.0 V)	7.06 ns out, 2.39 ns in @10 pF	VERIFIED	Re-parsed from the PDK liberty by the lead; matches the agent within 0.15 ns
HyperBus per port	90.4 MB/s · 8.2/ pin (aclk 50)	VERIFIED	Re-simulated by the lead; 0 byte errors; analytic PHY model 93.5 minus 1.1 % DMA. A low-SCK "hang" found during hardening was a testbench reset artifact (no falling edge on async reset); fixed, and headline numbers unchanged to 0.03 MB/s
HyperBus at ss today	73.5 MB/s (aclk 40); worst case 64.0	VERIFIED	New lead simulations
QSPI	24–25 MB/s @50 MHz	VERIFIED	Arithmetic re-derived. Needs opposite-edge capture and ≤10 pF
Streaming complex aclk @ ss	40.7 MHz	VERIFIED	Checked against the raw OpenSTA log (WNS −4.544 ns)
TCM read path	97.3 / 67.8 / 112 MHz (tt/ss/ff)	MEASURED	Routed stand-in, 3-corner STA, lead
4 ports on the pins	~300 MB/s at ss today; 380 ss / 588 tt with fixes; 728–754 only with unclosable 100 MHz clocks	VERIFIED	Byte-checked 4-port sim with an NPU-slave sink model; P&R timing per corner; lead re-ran 100/100 and ss-clock points
End-to-end decode (TinyStories-28M)	1.65 tok/s @50 MHz (2.78	VERIFIED	Full-loop RTL sim (HyperRAM model → iDMA → DTCM → GEMV), outputs

	with 8k vocab); 1.35 @40		checked; lead re-run cycle-identical on all 8 runs
GEMV ceiling	1.94 MAC/cycle = 97 MB/s (16×1024); 1.71 = 85 MB/s (64×256)	VERIFIED	Lead re-run cycle-identical on all 28 kernel/shape/data cases, all exact vs host golden; gate-level FPU-fixed run gives the same 8,466 cycles
VME engine	15.2 mm² (tile store 9.80)	VERIFIED	Lead re-synthesized the dominant tile store from scratch: 9,796,327 µm² and 32,824 flops, identical to the engineer's report; engine total = 5.40 + 9.80; flop delta vs TSMC 65 within 10 %
WS 16×16 array area	2.00 mm² cells, 32,598 flops	VERIFIED	Lead re-synthesized through an independent front end (sv2v + Yosys reader vs yosys-slang): 2.000 vs 2.002 mm², identical flop count, no latches
WS 16×16 array timing	routed 3.65 mm²; 49.8 / 24.0 / 61.8 MHz tt/ss/ff	MEASURED	Route 0 DRC; three-corner SPEF signoff by the lead. Hold not yet clean; RTL function from one engineer's testbench
WS prefill gain	≈ 61× on a 64-token prompt	PROJECTED	From measured utilization M / max(M, 47); the AXI wrapper and sequencer RTL is not written
Plan A die use	89 % of 32.2 mm²	PROJECTED	Cell area × 1.35 ÷ 0.60; run 1 suggests core + TCM needs about 20 mm², which would make it about 93 %
Streaming complex area	0.84 mm² placed (lean)	MEASURED	One P&R run, 0 DRC
Core Fmax, as released	22 MHz tt, post-CTS	MEASURED	Placement parasitics; routed SPEF and ss/ff pending
Core Fmax, FPU fixed	32.8 MHz tt, post-CTS	MEASURED	Runs 2 and 3 agree (32.8 / 32.3 MHz) in two different floorplans; placement parasitics; routed SPEF and ss/ff pending

			pending
Routed core block: area, DRC, IR drop, power, ss Fmax	not obtained	OPEN	Global route did not converge at 16.4, 17.6 or 21.1 mm² (6, 7 and 13 h). The routed WS array, streaming complex